

Software Architecture and Design Specification

Project: Leave Management System

Version: 1.0

Authors: Deepthi, Samrudhi, Hita, Chirag (LeaveItToUs)

Date: 28-09-2025

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	28 - 09 - 2025	LeaveItToUs	

Approvals

Role	Name	Signature/Date
Project Lead	Deepthi M	Deepthi 28-09-2025
QA Lead	Hita R	Hita 28-09-2025
Security Lead	Samrudhi Rao	Samrudhi 28-09-2025

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the **Leave Management System**. It translates requirements from the SRS into architectural artifacts, component responsibilities, interfaces, and design decisions to guide implementation, testing, security review, and maintenance.

1.2 Scope

This document covers the architectural design of the system's primary components, which were identified in the SRS and targeted for testing in the Software Test Plan (STP). The scope includes the design of:

- **Employee Services:** Architecture for modules supporting leave application, leave status tracking, and cancellation.
- **Manager Services:** Design for workflows handling review, approval, and rejection of employee leave requests.
- **Administrative Services:** Modules for maintaining leave records, generating reports, and ensuring compliance with organizational leave policies.

1.3 Audience

Project Managers, Developers, Test and QA engineers, system administrators and maintenance teams.

1.4 Definitions

- LMS(Leave Management System)
- HR(Human Resources)
- UI(User Interface)
- DBMS(Database Management System)
- API (Application Programming Interface)

2. Document Overview

2.1 How to use this document

This document provides the key architectural and design specifications for the **Leave Management System (LMS)**. It should be used as the primary technical blueprint by the development and testing teams. It begins with the architectural goals and guiding principles for building a scalable and policy-compliant system, then details the individual modules such as employee services, manager services, and administrative services. The design section provides UML diagrams, system workflows, data models, and use case descriptions, along with API definitions to ensure consistent implementation and integration across components.

2.2 Related Documents

SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirements Traceability Matrix), User Manual, and Design Document.

3. Architecture

3.1 Goals & Constraints

Goals:

- Secure handling of personnel and leave data
- Accurate leave balance calculations and audit trails
- High availability during working hours (target: 99% uptime)
- Extensible policy engine to support accrual, carry-forward, encashments

Constraints:

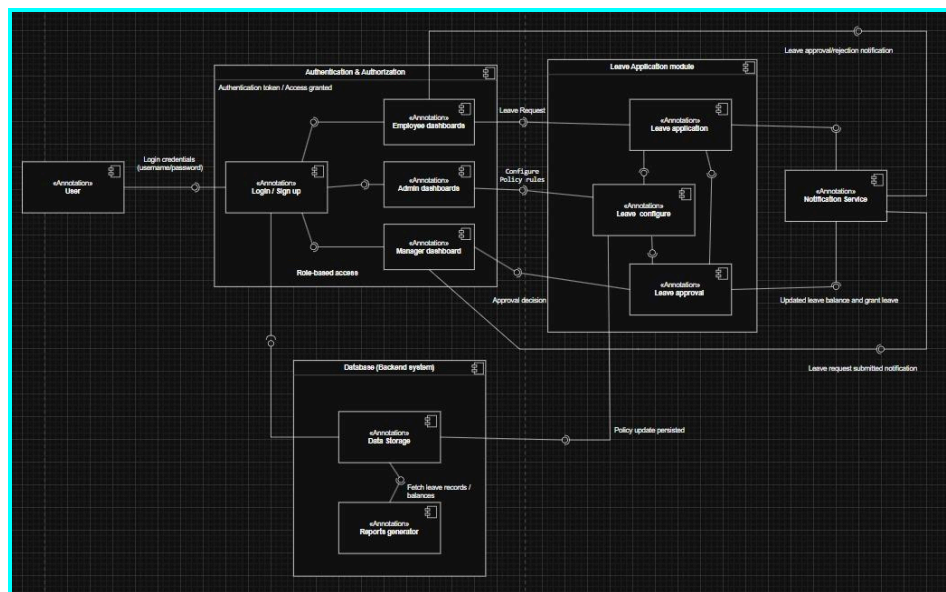
- Must enforce RBAC and least privilege
- Data privacy regulations and company policies apply
- Integration with existing email providers
- Limited initial scale: ~500 employees and 20 concurrent users (expandable)

3.2 Stakeholders & Concerns

The architecture addresses the primary concerns of the system's key stakeholders.

- **Employees:** Require a user-friendly interface, privacy safeguards, and accurate tracking of leave balances.
- **Managers:** Need timely approval workflows, visibility into team calendars, and clear notifications.
- **HR:** Depend on configurable policy settings, comprehensive reporting, and audit-ready records.
- **IT/Ops:** Require maintainable systems with effective monitoring, backup, and recovery capabilities.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- **Web UI / Mobile UI:** React-based dashboard for employees, managers, HR. Handles forms, calendars, and role-specific views.
- **API Gateway / Backend API:** RESTful endpoints (Node.js/Express) that validate requests and orchestrate services.
- **Auth Service:** Authentication and authorization (JWT). Performs RBAC enforcement and logs auth events.
- **Leave Processor:** Core business logic for applying leaves, validating dates, checking balances, applying accrual and carry-forward rules.
- **Policy Engine:** Stores and evaluates leave types, accrual rules, encashment, and business rules; configurable by Admin.
- **Notification Service:** Asynchronous notifications via email to ensure real-time updates.
- **Reporting Service:** Generates on-demand and scheduled reports (PDF/Excel) and supports filters and exports.
- **Database:** Relational DB (MySQL) for core data; audit logs and ledger tables for traceability.
- **Integration Connectors:** Holiday Calendar API.

3.5 Chosen Architecture Pattern and Rationale

A **Layered Modular Architecture** was chosen for the backend, consisting of an API layer, service layer, and data layer.

Rationale: This pattern was selected to balance modularity with ease of deployment. It provides:

1. **Modularity & Maintainability:** Clear separation of concerns makes the system easier to understand, extend, and maintain.
2. **Simplicity:** Keeps the initial deployment straightforward, reducing operational overhead.
3. **Scalability:** Specific services such as **Notifications** and **Reporting** can be scaled independently when needed.

3.6 Technology Stack & Data Stores

- Frontend: React, accessible components, responsive design
- Backend: Node.js + Express
- Database: MySQL (primary)
- Messaging: email service for notification
- Reporting: server-side PDF/Excel libraries

3.7 Risks & Mitigations

Risk: System downtime due to server failure.

- **Mitigation:** Deploy backup server and perform regular data backups.

Risk: Unauthorized access to employee leave records.

- **Mitigation:** Enforce role-based access control and encrypt sensitive data.

Risk: Data loss or corruption.

- **Mitigation:** Implement automated database backups and establish recovery procedures.

Risk: Policy misconfiguration (wrong leave rules applied).

- **Mitigation:** Conduct regular audits and require Admin validation of the policy engine.

3.8 Traceability to Requirements

Requirement ID	Requirement Description	Architecture Component(s) Involved
R1	User login & authentication	Authentication Service, RABC Module
R2	Apply for leave	LMS UI(Employee Module), Leave Management service
R3	Approve/Reject leave	Manager Module (UI), Leave Management Service
R4	Track leave balance/status	LMS UI (Employee Dashboard), Database (Leave Ledger)
R5	Generate reports	HR Console, Reporting & Audit Service, Database

3.9 Security Architecture

The architecture uses the STRIDE model to address threats identified in the security requirements.

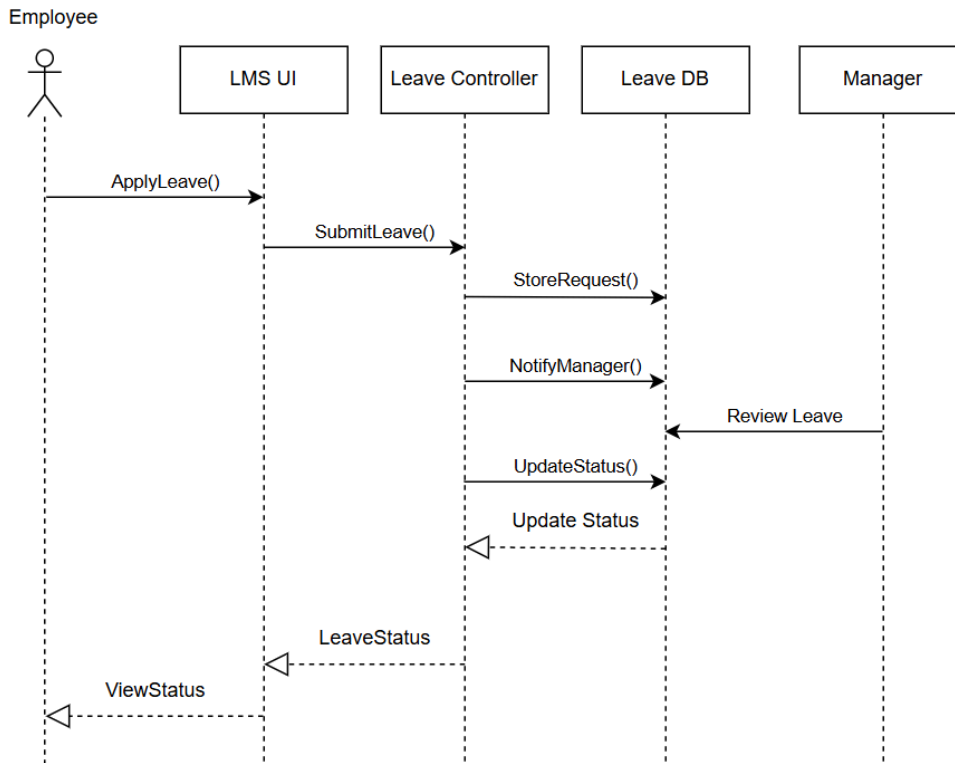
- **Spoofing:** Mitigated by strong authentication using SSO and MFA, along with session protections.
- **Tampering:** Input validation, integrity checks, and database permissions ensure data integrity.
- **(Repudiation)Audit:** immutable audit logs for critical actions (apply, approve, cancel, policy changes).
- **Information Disclosure:** TLS secures data in transit, sensitive data at rest is encrypted, and need-to-know access is enforced.
- **Denial of Service (DoS):** The system restricts too many requests and can adjust resources to keep running smoothly.
- **Elevation of Privilege:** RBAC is enforced server-side, and admin actions are guarded and logged.

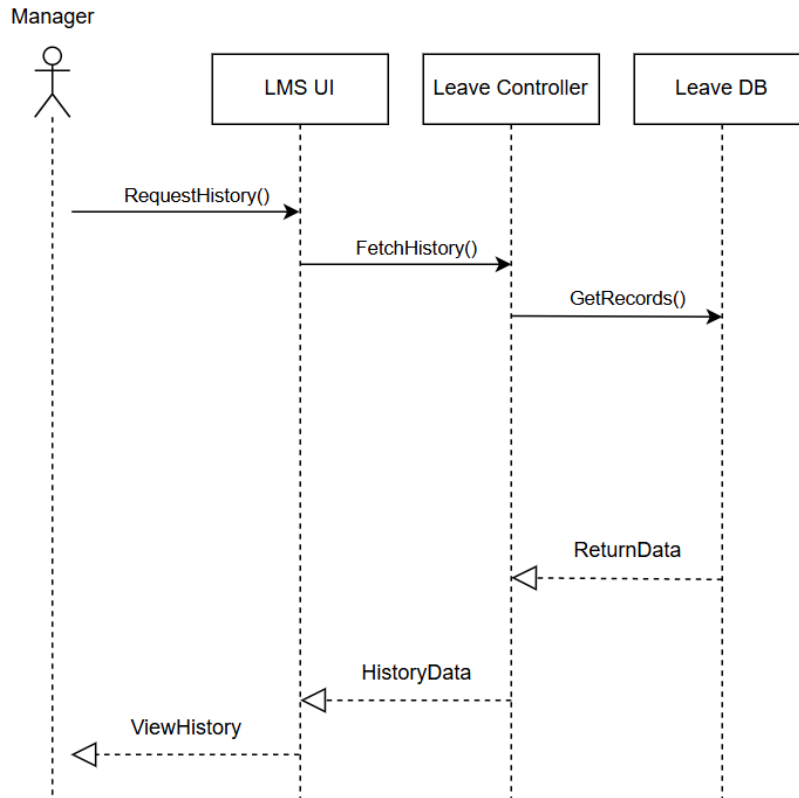
4. Design

4.1 Design Overview

LMS designed with layered modules for separation of concerns and easier maintenance.

4.2 UML Sequence Diagrams





4.3 API Design

Interface definition for at least 2 components.

- Endpoint: `/applyLeave`:

Method: POST

Request: `{employeeId, leaveType, startDate, endDate, reason}`

Response: `{status, leaveId, message}`

Errors:

401 Unauthorized employee

403 Leave quota exceeded

409 Overlapping leave request

- Endpoint: `/approveLeave`:

Method: POST

Request: `{managerId, leaveId, decision}`

Response:{status, leaveId, message}

Errors:

401 Unauthorized manager

404 Leave request not found

409 Leave already processed

- Endpoint: /leaveHistory:

Method: GET

Request:{employeeId}

Response:{employeeId, leaveId, leaveType, startDate, endDate, status}

Errors:

401 Employee Unauthorized

4.4 Error Handling, Logging & Monitoring

Standardized error messages. No sensitive employee information in logs. Monitoring includes leave request failure rate, approval turnaround time, and system uptime.

4.5 UX Design

Employee Dashboard: Leave balance, request form, leave history, status tracking.

Manager Dashboard: Pending approvals, team leave calendar, approval/rejection options.

HR Dashboard: Reports, policy management, employee leave trends.

General Features:

- Role-based access (Employee, Manager, HR).
- Leave calendar with color-coded statuses (approved, pending, rejected).
- Notifications (email/app alerts).

4.6 Open Issues & Next Steps

Current Limitations: No integration with payroll/attendance yet.Limited reporting functionality.

Future enhancements: AI-based leave approval recommendations, integration with biometric attendance, and chatbot support for employee leave queries.

5. Appendices

5.1 Glossary:

- **API (Application Programming Interface):** Enables communication between LMS front-end and back-end.
- **RBAC (Role-Based Access Control):** Security model ensuring employees, managers, and HR have appropriate access.
- **JWT (JSON Web Token):** Secure token format used for employee login and session management.
- **Leave Balance:** Remaining number of leave days available for an employee.
- **Leave Quota:** Maximum number of leave days allocated per leave type (casual, sick, earned).
- **Approval Workflow:** The process of manager reviewing and approving leave requests.
- **Carry Forward:** Unused leave days that can be transferred to the next year.

5.2 References:

IEEE 830/ 29148,ISO 27001, NIST,OWASP ASVS

5.3 Tools:

PlantUML, draw.io, Swagger,Postman,OWASP ZAP,MySQL Workbench,Node.js + Express,React